Vani Integration — Scope Review: Alerts Only, or the Whole Machine?

Question (Deepak, 5 Nov 2026): is the integration the platform team has planned restricted to **alerts** — or should it cover all the machine features: **machine, health, prediction, operation**?

> **DECISION — Deepak Panguluri, 5 Nov 2026: the four-lane scope in §3 is ADOPTED as the integration's target.** Lane A (alerts) proceeds exactly as the team has planned and built. Lane C's Phase 6 ask tool is **approved as built**. Lane D proceeds on the **acts** basis defined there: Vani acts only on the customer's explicit words on the call, confirmed back by Vani before she closes. The team's Phase 0 answers (sent separately) are unchanged.
>
> **Document access:** this memo is part of the team handoff bundle (with the Phase 0 answers, the Integration Guide and the Machine Lane Spec, plus a bundle README mapping every reference). If you received this file alone, ask Deepak for the bundle; committing the bundle into the team repo (`docs/ai/vani/`) makes every reference in it resolve permanently.

Checked: the team's *Vani Voice Integration Plan* (branch `feature/vani-voice`, 4 Nov) against Deepak's *Machine Lane Spec v0.1* (3 Nov 2026) — the vision the integration exists to serve.

Verdict

Yes — as planned, the integration is alerts-only in practice. Health and prediction survive only as *text fields inside an alert packet*, and operation (work orders, incidents, service actions) is essentially absent. That is narrower than the Nov 3 vision in four specific ways (§2). The good news: everything the team has built stays — the full scope is an **extension of the same contract**, not a rebuild (§3).

1. Coverage matrix — team's plan vs the four feature areas

Feature area	Outbound (Vani calls the customer)	Inbound (customer calls Vani)
Machine (registry, status, lifecycle)	⚠️ Partial — machine identity rides in the alert packet; machine added/removed pushes a snapshot. But machine *state changes* that are not alerts (went offline, stopped, back online) never reach Vani.	⚠️ Snapshot only — "today an inbound caller hears only the machine snapshot" (the plan's own words). Machine list + health status, no depth.

Feature area	Outbound (Vani calls the customer)	Inbound (customer calls Vani)
Health (ThingsCare, health card, score trend)	⚠️ As a field — `health_card` is one text field in the alert packet. A health score sliding for a week fires **no call** unless an alert rule happens to fire.	⚠️ Status-level only in the snapshot; the health *card* (trend, watch band, what changed) is not in the snapshot contract.
Prediction (Live Predictions, scenarios, confidence)	❌ **Alerts only.** The Nov 3 spec's trigger had two kinds — `alert` **and** `trigger_action` — and predictions were first-class content with confidence. The plan's outbox hooks the **alert notifier only**: a prediction that has not (yet) raised an alert can never earn a call, and confidence is flattened into the summary text.	⚠️ Only via Phase 6 (below).
Operation (work orders, incidents, service)	❌ **Absent.** No operational triggers (service due, work order completed, incident escalated), and — bigger — **no return path for what the customer decides on the call**: "yes, schedule the inspection" is recorded as a call *outcome*, but nothing on the platform side turns it into a work order or an acknowledgement. The loop never closes.	⚠️ Phase 6's ask tool can *read* work orders via the assistant — read-only, and gated off (below).

And Phase 6 — the one part of the plan that *does* reach health, prediction and operation (the ask-the-platform endpoint over the assistant's tools: `machine_page`, `list_alerts`, `logger_health`, `list_work_orders`…) — is built but **switched off pending Deepak's agreement**, because it changes the "packet only" promise. So even the broad part of the plan is parked, and what is *scheduled to go live* is the alert lane alone.

2. Where the plan narrowed the Nov 3 vision

1. **Trigger kinds:** spec §3.2 = `alert` **or** `trigger_action` ("alerts **or based on trigger actions** from the IoT platform" — Deepak's words). Plan = alert events only, hooked beside the email notifier.
2. **Inbound review:** spec §4 = a first-class flow — the fork question, named-machine depth (health card, latest predictions *with confidence*, recent alerts and what became of them), and an account review walk in priority order. Plan = snapshot greeting; depth deferred to the gated Phase 6.

3. **Packet richness:** spec's machine packet is a structured object (health summary, prediction **statement + confidence**, signals with thresholds, notes/history). Plan's packet flattens these into speakable text fields — acceptable for v1 alert calls, lossy as the *only* contract: confidence and thresholds are exactly what a good machine conversation needs.
4. **Informs vs acts — still undecided since Nov 3:** the vision's open prerequisite was whether Vani only *informs* or also *acts* (acknowledge alarms, open ThingsCare tickets/work orders, escalate). The plan silently chose "inform" (outcomes are records, not actions). That may be right for v1 — but it should be a **recorded decision**, not a default nobody chose.

3. Recommended full scope — same contract, four lanes (nothing rebuilt)

Lane	Content	What it reuses / adds
A · Alerts (as planned)	Team Phases 0–5 unchanged. First to go live.	Already built on the branch.
B · Predictions & trigger actions	Restore `trigger.kind = trigger_action`; let Live Predictions fire triggers when a prediction crosses its risk/confidence line — the packet already has the prediction field; add `confidence` as its own field. Health-band crossings (score enters watch/critical band, or drops N points in M days) fire as trigger actions too — **health becomes a trigger source, not just a text field.**	Same endpoint, same idempotency, same severity policy (Deepak's rules evaluate all kinds). Platform work: emit triggers from Predictions/ThingsCare, not only the alert notifier.
C · Inbound full review	Decide Phase 6 (recommendation: **approve the ask tool as built** — one question endpoint, caller-scoped, answer-only, refusals honest). Enrich the snapshot in parallel: full health card + open predictions + open work orders per machine, so the greeting/review works even before a question is asked.	Phase 6 is **already built**; the snapshot is a payload extension, not a new endpoint.

Lane	Content	What it reuses / adds
D · Operation loop (the acts decision)	Extend the outcome callback with a `customer_decision` (e.g. `inspection_approved`, `call_back_requested`, `declined`, `acknowledged`) so the platform can act: acknowledge the alert, create the work order, schedule the follow-up. Operational triggers join Lane B (service due, work order awaiting customer confirmation).	One added callback field + platform-side handlers. **This lane needs Deepak's informs-vs-acts decision before build** — recommendation: acts, but only on explicit customer words on the call, always confirmed back by Vani before she closes.

Sequencing: A goes live first (acceptance test as planned) → C's decision can land the same week (code exists) → B next (small platform addition, high value: predictions are the platform's differentiator — an integration that can't call about a prediction until it becomes an alert wastes it) → D when Deepak decides informs-vs-acts.

4. Admin panel — per-customer feature selection for Vani calls (Deepak, 5 Nov 2026)

The four lanes are not all-or-nothing. **In the Vani Admin panel, every customer account carries a feature selection: which feature areas Vani may call about — and discuss — for that customer.** One customer may want alert calls only; another wants prediction and health calls too, with operation off. Deepak sets this at registration and can change it any time; it is an admin function, not a code change.

Where it lives: Admin panel → Vani → **Customers** (registry manager), one feature block per account — alongside the account's severity floor, quiet hours and caps (it extends the registry's existing `alert_preferences`). The **Rules editor** holds the global defaults a new account starts from.

The options (per account):

Feature toggle	When ON, Vani may…	Default
Machine alerts (Lane A)	Call when a machine alert fires for this account	ON
Health (Lane B/C)	Call on health-band crossings (score enters watch/critical band, sharp drop); use the full health card inbound	OFF until set
Prediction (Lane B/C)	Call when a prediction / trigger action crosses its risk line; discuss open predictions with confidence inbound	OFF until set

Feature toggle	When ON, Vani may…	Default
Operation (Lane D)	Call on operational triggers (service due, work order awaiting confirmation); act on the customer's decisions (acknowledge / create work order)	OFF until set

The same toggles scope the **inbound** conversation: a feature that is OFF for an account is not discussed either — Vani answers from the enabled features only and says so plainly if asked beyond them.

Behaviour rules:

- The feature check runs **first** in the route-rules engine, before severity floor, quiet hours and caps. A trigger whose feature is OFF for the account is still accepted, logged and reported — outcome `suppressed_by_policy`, reason **`feature_not_enabled`** — so the platform always knows why no call happened.
- Snapshots keep flowing for every registered machine regardless of toggles: switching a feature ON never starts from stale data.
- Per-account selection overrides the global default; the panel shows both the default and the account's effective selection.
- Changing a toggle takes effect on the next trigger — no restart, no deploy (it is registry data, like the rest of the calling policy).

Data shape (for the build): one field on the account registration — `features: { alerts: true, health: false, predictions: false, operations: false }` — plus the trigger request naming its feature kind (Lane B already adds `trigger.kind`; the feature is derived from it: alert → alerts, prediction/trigger-action → predictions, health-band event → health, service/work-order event → operations).

Acceptance (add to the trial list): with `predictions` OFF on `usa-test-01`, a prediction trigger returns `suppressed_by_policy / feature_not_enabled` and no call; an alert trigger on the same account still dials. Toggle predictions ON and the same prediction trigger dials.

5. What does NOT change

Phase 0 answers stand as sent (token, `high → warning`, callback secret, Railway variables). The registry, severity policy, caps, quiet hours and grounding rules are identical across all four lanes — the lanes differ only in **what may fire a trigger** and **what the conversation may draw on**, never in who may be called or what Vani may invent (nothing).

Decisions — recorded 5 Nov 2026: (1) four-lane scope **adopted**; (2) Phase 6 ask tool **approved as built**; (3) informs vs acts — **acts**, on the customer's explicit words only, confirmed back on the call (Lane D basis).